

Rapport de projet

Gestion de Pharmacie

Réalisé par :

Houda Jardak (G1)

Alaa Eddine Fadhel(G1)

May Abdallah (G2)

Nouhe Jedidi (G3)

Malek Gdara (G3)

I-Introduction :

Dans le cadre de ce projet académique, nous avons réalisé une application de gestion de pharmacie visant à faciliter la gestion des médicaments, des fournisseurs, des clients, du stock et des opérations associées. Malgré une durée limitée à une semaine, une méthodologie rigoureuse a été adoptée afin d'assurer la qualité du produit final, la cohérence de la conception et l'efficacité du travail en équipe. Le projet a été développé en mettant l'accent sur une conception solide en amont, une architecture claire et une bonne organisation du travail collaboratif.

II-Organisation et méthodologie adoptée :

Étant donné la courte durée du projet, une organisation par phases a été retenue plutôt qu'un découpage en sprints classiques. Cette approche a permis de structurer le travail de manière efficace, tout en respectant les contraintes temporelles et pédagogiques. Les phases du projet ont été définies de manière progressive, allant de la conception à l'implémentation, puis à la finalisation et aux tests.

1)Phase de conception et préparation :

a-Analyse des besoins fonctionnels

La première phase a été consacrée à l'analyse des besoins du système. Cette étape a permis d'identifier les fonctionnalités principales attendues d'un système de gestion de pharmacie, telles que :

- la gestion des médicaments
- la gestion des fournisseurs et des clients
- le suivi du stock
- la gestion des ventes et des commandes
- la visualisation globale via un tableau de bord.

Cette analyse a servi de base pour la conception de la base de données et de l'architecture logicielle.

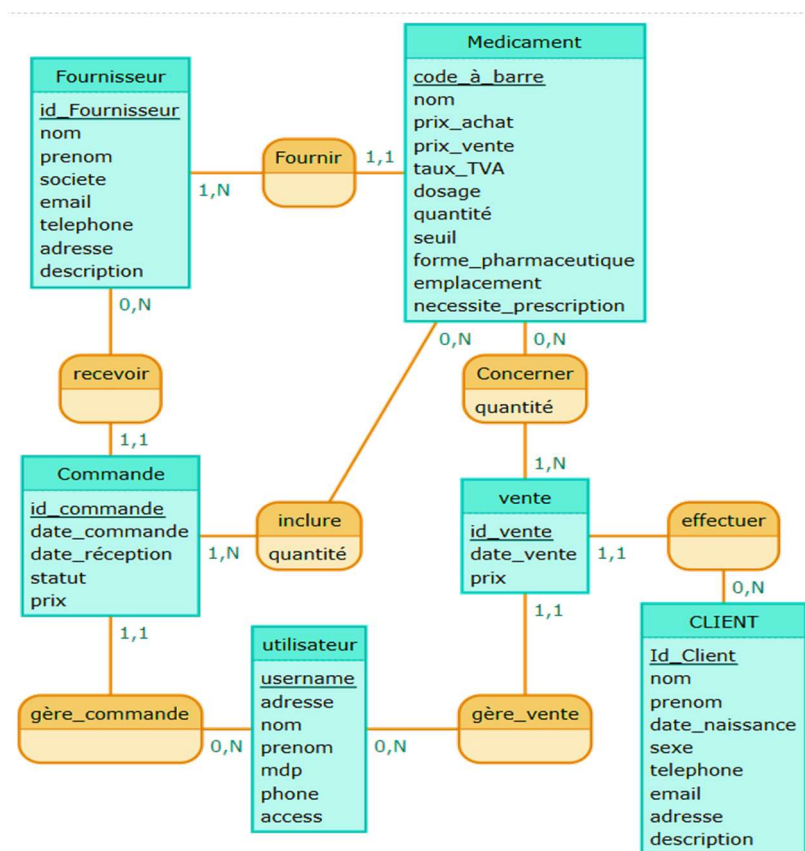
b-Conception de la base de données :

Une importance majeure a été accordée à la conception de la base de données afin d'assurer l'intégrité, la cohérence et la fiabilité des données manipulées par l'application. Cette phase a inclus :

- l'élaboration du Modèle Conceptuel de Données (MCD)
- la transformation du MCD en Modèle Logique de Données (MLD)
- l'application des règles de normalisation afin d'éviter les redondances
- la définition précise des clés primaires et étrangères

Le modèle de données obtenu respecte la **troisième forme normale (3FN)**, garantissant l'absence de redondances, la dépendance fonctionnelle correcte des attributs et une meilleure cohérence des données.

Le schéma relationnel final a ensuite été implémenté à l'aide de scripts SQL, garantissant une base de données bien structurée et adaptée aux besoins du système.



Fournisseur (id_Fournisseur, nom, prénom, société, email, téléphone, adresse, description)

Commande (id_commande, date_commande, date_réception, statut, prix, #id_fournisseur, #username)

Médicament (code à barre, nom, prix_achat, prix_vente, taux_TVA, dosage, quantité, seuil, forme_pharmaceutique, emplacement, necessite_prescription, #id-Fournisseur)

CommandeProduit (#code à barre, #id_commande, quantité)

VenteProduit (#code à barre, #id vente, quantité)

Vente(id vente, date_vente, prix, #Id_client, #username)

Utilisateur (username, adresse, nom, prénom, mdp, phone, access)

Client(Id_client, nom, prénom, date_naissance, sexe, téléphone, email, adresse, description)

c- Conception de l'architecture logicielle et modélisation UML :

Avant de commencer le développement, la structure globale du projet a été définie afin de garantir une séparation claire des responsabilités. Le projet a été organisé selon une architecture inspirée des principes MVC et du patron DAO. Les packages ont été organisés comme suit :

- model : représentation des entités du système
- dao : gestion de l'accès aux données et des opérations sur la base de données
- ui : interfaces graphiques destinées à l'utilisateur, implémentation de la logique métier
- util : classes utilitaires, notamment pour la gestion de la connexion JDBC et la session de l'utilisateur.

Cette phase a également permis d'aboutir à un **diagramme de classes UML**, établi à partir d'une cohérence rigoureuse entre les entités métier et la base de données. Ce diagramme a facilité la compréhension des relations entre les différentes classes du système et a servi de référence tout au long de l'implémentation.

2) Phase d'implémentation :

a- Introduction de la phase d'implémentation :

La phase d'implémentation a consisté à développer les différents modules de l'application conformément à la conception établie. Afin d'optimiser le travail en groupe, les tâches ont été réparties entre les membres de l'équipe. Dans un premier temps,

l'équipe s'est concentrée sur les modules de base du système, à savoir la gestion des fournisseurs, des médicaments, des employés et des clients.

Chaque membre a pris en charge un module spécifique du système et a assuré l'implémentation complète des couches correspondantes, incluant les classes modèle, DAO et interfaces graphiques. Une fois ces parties fondamentales stabilisées et fonctionnelles, l'équipe a procédé à la finalisation des modules plus complexes, notamment la gestion des commandes, la gestion des ventes, ainsi que le tableau de bord (dashboard).

La connexion à la base de données a été assurée via JDBC, permettant une interaction sécurisée avec la base de données à l'aide de PreparedStatement pour l'exécution des opérations CRUD. Une attention particulière a été portée à la cohérence entre la base de données, la logique métier et les interfaces graphiques afin d'assurer un fonctionnement fiable et fluide de l'application.

b- Gestion collaborative et coordination du projet :

Pour assurer une bonne coordination, l'équipe a utilisé GitHub comme outil de gestion de versions. Chaque membre effectuait des push réguliers de son travail vers le dépôt distant. Des réunions quotidiennes ont été organisées afin de suivre l'avancement du projet, effectuer les fusions nécessaires et résoudre les éventuels conflits de version.

c- Interfaces graphiques et modules :

L'application propose une interface graphique structurée autour de plusieurs modules fonctionnels, avec une gestion différenciée des accès selon le type d'utilisateur. Deux niveaux d'accès ont été définis afin de renforcer la sécurité et l'organisation du système :

- **Accès administrateur (Admin)** : accès exclusif à certains panneaux, notamment la gestion des employés et le tableau de bord, permettant une supervision globale de l'activité.
- **Accès standard** : accès aux modules de gestion des fournisseurs, des clients, du stock, des commandes et des ventes.

Interface de login :

- **Classe** : LoginPanel
- **Description** : Première interface à apparaître au lancement du projet. L'utilisateur doit saisir son nom d'utilisateur et son mot de passe. La vérification est effectuée via la base de données, et un message d'erreur apparaît si les identifiants sont incorrects.
- **Fonctionnalité** : Sécurisation de l'accès à l'application.

Tableau de bord (Dashboard) :

- **Classe** : DashboardPanel

- **Description** : Interface de visualisation globale permettant d'analyser l'activité de la pharmacie à travers plusieurs indicateurs clés, notamment le chiffre d'affaires, les performances des employés, l'efficacité des fournisseurs, l'état du stock des médicaments ayant atteint le seuil critique, ainsi que le suivi global des ventes et des commandes.

	model	dao	ui	Fonctionnalité
Modules clients	Customer	CustomerDao	AddCustomerPanel, AllCustomerPanel, SearchCustomerPanel, CustomerPanel	Ajouter, visualiser, rechercher et gérer les clients.
Modules fournisseurs	Supplier	SupplierDao	AddSupplierPanel, AllSupplierPanel, SearchSupplierPanel, SupplierPanel	Gestion complète des fournisseurs.
Modules employés	Employee	EmployeeDao	AddEmployeePanel, AllEmployeesPanel, SearchEmployeePanel, EmployeePanel	Gestion des employés et contrôle des accès.
Modules médicaments	Medicine	MedicineDao	AddMedicinePanel, AllMedicinePanel, SearchMedicinePanel, MedicinePanel, ModifyByMedicine.	Gestion des médicaments et suivi du stock.
Modules commandes	Command, CommandItem	CommandDao, CommandItemDao	CommandPanel, ViewCommandPanel	Création et consultation des commandes.
Modules ventes	Sale, SaleItem	SaleDao, SaleItemDao	SalePanel, ViewSalePanel	Gestion des ventes et suivi des articles vendus.

Gestion de la connexion :

- **Package** : util → DatabaseConnection, session

- **Fonctionnalité** : Gestion de la connexion JDBC sécurisée à la base de données, ainsi que la gestion de la session utilisateur permettant de conserver les informations de l'utilisateur connecté, de vérifier l'état de connexion et de contrôler les accès aux différentes interfaces de l'application.

3) Phase de finalisation :

Gestion des exceptions :

Une gestion appropriée des exceptions a été mise en place afin d'améliorer la fiabilité, la robustesse et la sécurité de l'application. Des exceptions personnalisées ont été définies pour traiter les erreurs les plus fréquentes pouvant survenir lors de l'exécution du système, notamment celles liées à la saisie utilisateur et à la cohérence des données. Dans ce cadre, plusieurs classes d'exceptions spécifiques ont été implémentées :

- `CodeBarreExistsException` : cette exception est déclenchée lors de l'ajout d'un médicament lorsque le code-barres saisi existe déjà dans la base de données. Elle permet d'éviter la duplication des données et de garantir l'unicité des identifiants des médicaments.
- `DataMissingException` : cette exception est utilisée lorsque des informations obligatoires ne sont pas fournies par l'utilisateur lors de la saisie des formulaires. Elle assure la validation des données avant toute opération d'insertion ou de mise à jour dans la base de données.
- `InvalideUsernameException` : cette exception est levée lors du processus d'authentification, lorsque le nom d'utilisateur saisi est invalide ou ne respecte pas les règles définies. Elle contribue à sécuriser l'accès à l'application et à améliorer la gestion du module de connexion. L'utilisation de ces exceptions personnalisées a permis de mieux contrôler les erreurs, d'améliorer l'expérience utilisateur à travers des messages clairs et explicites, et de renforcer la stabilité globale de l'application.

III- Tests et validation :

Des tests ont été réalisés afin de garantir le bon fonctionnement de l'application :

- tests unitaires sur les classes DAO.
- tests fonctionnels sur les interfaces graphiques et la navigation.
- vérification de la cohérence et de l'intégrité des données dans la base de données.

IV- Difficultés rencontrées lors de la mise en œuvre :

Lors du développement de notre application Java, notre équipe a rencontré plusieurs défis techniques et organisationnels, certains résolus grâce à des stratégies efficaces, d'autres restant en cours de résolution.

1) Difficultés et problèmes résolus :

Organisation du travail et brainstorming :

Difficulté : Coordination difficile des tâches et des idées.

Solution : réunions quotidiennes et brainstorming, jusqu'à deux réunions les jours de forte charge.

Synchronisation du travail et suivi de projet :

Difficulté : Travail simultané sur différentes parties.

Solution : apprentissage de Git, création de branches sur GitHub et fusion régulière pour garder le projet cohérent.

Manipulation de la base de données :

Difficulté : Interfaçage Java-Base de données inconnu.

Solution : apprentissage de JDBC et des connecteurs SQL pour exécuter les opérations CRUD.

Développement de l'interface utilisateur (UI) :

Difficulté : Faible expérience avec Swing.

Solution : Suivi de tutoriels vidéo et utilisation de GitHub Copilot pour automatiser les tâches répétitives.

Organisation des packages, répertoires et classes :

Difficulté : Structurer le projet de manière cohérente.

Solution : Analyse des anciens projets et des GitHub repositories pour assurer une organisation efficace et un travail collaboratif.

V- Conclusion :

Ce projet a permis de mettre en pratique les notions de conception de bases de données, d'architecture logicielle, de modélisation UML et de développement d'applications Java. Malgré une durée limitée à une semaine, l'organisation rigoureuse, la conception préalable et la collaboration via GitHub ont permis de réaliser une application fonctionnelle, structurée et évolutive répondant aux besoins d'un système de gestion de pharmacie.